

INTRODUCTION



Welcome to *Impractical Python Projects*! Here, you'll use the Python programming language to explore Mars, Jupiter, and the farthest reaches of the galaxy; the souls of poets; the world of high finance; the underworld of espionage and vote tampering; the trickery of game shows; and more. You'll use techniques such as Markov chain analysis to write haiku, Monte Carlo simulation to model financial markets, image stacking to improve your astrophotography, and genetic algorithms to breed an army of gigantic rats, all while gaining experience with modules like `pygame`, `Pylint`, `pydocstyle`, `tkinter`, `python-docx`, `matplotlib`, and `pillow`. And most of all, you'll have fun.

Who This Book Is For

You can think of this as your second Python book. It's designed to follow and complement either a complete beginner's book or an introductory class. You'll be able to continue self-training using a project-based approach, without wasting your money or shelf space on a thorough rehashing of concepts you've already learned. But don't worry, I won't leave you hanging; all the code is annotated and explained.

These projects are for anyone who wants to use programming to conduct experiments, test theories, simulate nature, or just have fun. This includes people who use programming as part of their jobs (like scientists

and engineers) but who aren't programmers per se, as well as those I call the "determined non-specialists"—dilettantes and dabblers who enjoy programming problems as a fun pastime. If you've wanted to toy with the concepts presented here but found starting potentially complicated projects from scratch too daunting or time-consuming, this book is for you.

What's in This Book

As you work through the projects, you'll increase your knowledge of useful Python libraries and modules; learn more shortcuts, built-in functions, and helpful techniques; and practice designing, testing, and optimizing programs. Additionally, you'll be able to relate what you're doing to real-world applications, datasets, and issues.

To quote Ralph Waldo Emerson, "Nothing great was ever achieved without enthusiasm." This includes the learning experience. The ultimate goal of this book is to spark your imagination and lead you to develop interesting projects of your own. Don't worry if they seem too ambitious at first; a little diligence and a lot of googling can work miracles—and faster than you think.

The following is an overview of the chapters in this book. You don't have to work through them sequentially, but the easiest projects are at the beginning, and I explain new concepts, modules, and techniques more thoroughly when they're first introduced.

Chapter 1: Silly Name Generator This warm-up project introduces the Python PEP 8 and PEP 257 style guides as well as the Pylint and pydocstyle modules, which analyze your code's conformance to these guidelines. The end product is a goofy-name generator inspired by the USA Network TV show *Psych*.

Chapter 2: Finding Palingram Spells Learn how to profile your code while saving DC Comics sorceress Zatanna from a painful death. Search online dictionaries for the magical palingrams that Zatanna needs to beat a time-reversing villain.

Chapter 3: Solving Anagrams Write a program that helps a user create a phrase anagram from their name; for example, *Clint Eastwood* yields *old west action*. Then help Tom Marvolo Riddle derive his anagram, “I am Lord Voldemort,” using linguistic sieves.

Chapter 4: Decoding American Civil War Ciphers Investigate and crack one of the most successful military ciphers in history, the Union route cipher. Then help spies on both sides send and decode secret messages using the zig-zagging rail fence cipher.

Chapter 5: Encoding English Civil War Ciphers Read a message hidden in plain sight by decoding a null cipher from the English Civil War. Then save the head of Mary, Queen of Scots, by designing and implementing code to accomplish the more difficult task of *writing* a null cipher.

Chapter 6: Writing in Invisible Ink Help a corporate mole betray Sherlock Holmes’s dad and evade detection with invisible electronic ink. This chapter is based on an episode of the CBS television show *Elementary*.

Chapter 7: Breeding Giant Rats with Genetic Algorithms Use genetic algorithms—inspired by Darwinian evolution—to breed a race of super-rats the size of female bullmastiffs. Then help James Bond crack a safe with 10 billion possible combinations in the blink of an eye.

Chapter 8: Counting Syllables for Haiku Poetry Teach your computer to count syllables in English as a prelude to writing Japanese poetry, or haiku, in the next chapter.

Chapter 9: Writing Haiku with Markov Chain Analysis Teach your computer to write *haiku* by combining the syllable-counting module from **Chapter 8** with Markov chain analysis and a training corpus of several hundred ancient and modern haiku.

Chapter 10: Are We Alone? Exploring the Fermi Paradox Investigate the absence of alien radio signals using Drake’s equation, the dimensions of the Milky Way galaxy, and assumptions about the size of detectable

“emissions bubbles.” Learn and use the popular `tkinter` module to build a graphical display of the galaxy and Earth’s own radio bubble.

Chapter 11: The Monty Hall Problem Help the world’s smartest woman win the Monty Hall problem argument. Then use object-oriented programming (OOP) to build a version of Monty’s famous game with a fun graphical interface.

Chapter 12: Securing Your Nest Egg Plan your (or your parents’) secure retirement using a Monte Carlo–based financial simulation.

Chapter 13: Simulating an Alien Volcano Use `pygame` to simulate a volcanic eruption on Io, one of Jupiter’s moons.

Chapter 14: Mapping Mars with the Mars Orbiter Build a gravity-based arcade game and nudge a satellite into a circular mapping orbit without running out of fuel or burning up in the atmosphere. Display readouts of key parameters, track orbital paths, add the planet’s shadow, and spin Mars slowly on its axis, all while learning orbital mechanics!

Chapter 15: Improving Your Astrophotography with Planet Stacking Reveal Jupiter’s cloud bands and Great Red Spot by optically stacking poor-quality video images using the Python imaging library. Learn how to work with files, folders, and directory paths using the built-in `os` and `shutil` modules.

Chapter 16: Finding Frauds with Benford’s Law Use *Benford’s law* to investigate vote tampering in the 2016 presidential election. Use `matplotlib` to summarize the results in a chart.

Each chapter ends with at least one Practice Project or Challenge Project. Each Practice Project comes with a solution. That doesn’t mean it’s the *best* solution—you may come up with a better one on your own, so don’t peek ahead!

With the Challenge Projects, however, you’re truly on your own. When Cortez invaded Mexico in 1519, he burned his caravels so that his con-

quistadors would realize there was no going back; they would have to face the Aztecs with grim and unwavering determination. Thus, the expression “burn your boats” has come to represent wholeheartedness or full commitment to a task. This is how you should face the Challenge Projects—as if your boat were burned—and if you do, you’re likely to learn more from these exercises than from any other part of the book!

Python Version, Platform, and IDE

I constructed each of the projects in this book with Python v3.5 in a Microsoft Windows 10 environment. If you’re using a different operating system, no problem: I suggest compatible modules for other platforms, where appropriate.

The code examples and screen captures in this book are from either the Python IDLE text editor or the interactive shell. IDLE stands for *integrated development and learning environment*. It’s an *integrated development environment (IDE)* with an *L* added so that the acronym references Eric Idle of *Monty Python* fame. The interactive shell, also called the *interpreter*, is a window that lets you immediately execute commands and test code without needing to create a file.

IDLE has numerous drawbacks, such as the lack of a line-number column, but it is free and bundled with Python, so everyone has access to it. You are welcome to use whichever IDE you wish. There are many choices available online, such as Geany (pronounced *genie*), PyCharm, and PyScripter. Geany works with a wide range of operating systems, including Unix, macOS, and Windows. PyCharm works with Linux, Windows, and macOS. PyScripter works with Windows. For an extensive listing of available Python development tools and compatible platforms, visit <https://wiki.python.org/moin/DevelopmentTools/>.

The Code

Every line of code is provided for each project in this book, and I recommend you enter it by hand whenever possible. A college professor once

told me that we “learn through our hands,” and I have to agree that keying in code forces you to pay maximum attention to what’s going on.

But if you want to complete a project quickly or you accidentally delete all your work, you can download all of the code, including solutions to the Practice Projects, from <https://www.nostarch.com/impracticalpython/>. This site also contains the book’s errata sheet, in the event of future updates or changes.

Coding Style

This book is about problem solving and beginner-level fun, so the code may deviate at times from best practices and peak efficiency.

Occasionally, you may use list comprehension or a special operator, but for the most part, you’ll focus on simple, approachable code that’s easy to learn.

Keeping things simple is important for the programming nonprogrammers who read this book. Much of their code may be “Kleenex code”—used once or twice for a specific purpose and then thrown away. This is the type of code that might be shared with colleagues, or thrust upon them during staff changes, so it should be easy to pick up and understand.

All of the main project code is annotated and explained in a stand-alone manner, and it generally follows the style recommendations from *Python Enhancement Proposal 8*, otherwise known as *PEP 8*. Details on PEP 8, and software to help you honor these guidelines, are in **Chapter 1**.

Where to Get Help

Taking on a programming challenge can be, well, challenging. Coding isn’t always something that you can intuitively figure out—even with a language as friendly as Python. Throughout the following chapters, I will provide links and references to useful sources of information, but for projects you formulate on your own, nothing can beat online searches.

The key to successful searching is knowing what to ask. This can be quite frustrating at first, but think of it as a game of Twenty Questions. Keep honing your keywords with each successive search until you find an answer or reach a point of diminishing returns.

If books and online searches fail, then the next step is to ask someone. You can do this online, either for a fee or at free forums like Stack Overflow (<https://stackoverflow.com/>). But be warned: the members of these sites don't suffer fools gladly. Be sure to read their "How do I ask a good question?" pages before posting; for example, you can find the one for Stack Overflow at <http://stackoverflow.com/help/how-to-ask/>.

Onward!

Thanks for taking the time to read the Introduction! You clearly want to get as much as possible from this book, and you're off to a good start. When you reach the other end, you'll be more adept at Python and better prepared to solve challenging real-world problems. Let's get to work.